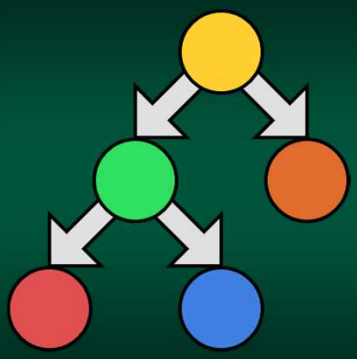
Binary Search Trees

Part 11

1



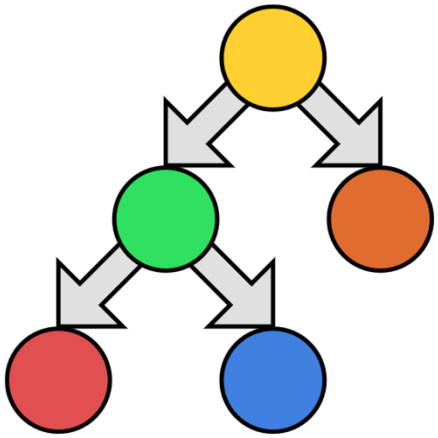
Binary Search Trees

Climbing a tree has never been so fun

2

Binary Search Trees

- *Binary Search Tree (BST)* is a special type of binary tree that sorts nodes by value
- For each node…
  - all the nodes on the left branch are less than it
  - all the nodes on the right branch are greater than it



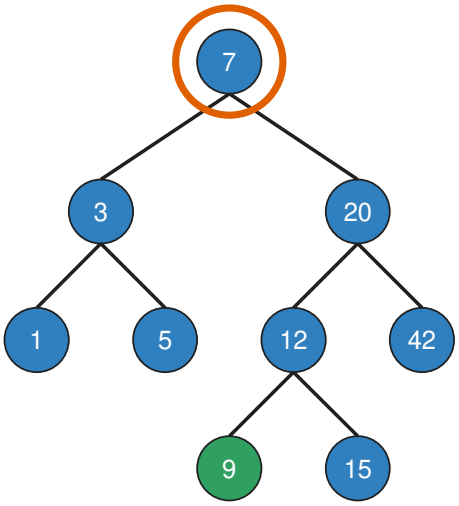
Searching the Tree

- Since the tree divides the problem progressively by two, the time complexity is only O(log n)
- Which gives that all the benefits of a sorted array
- Worst case is O(n) – if the tree is a list-like chain

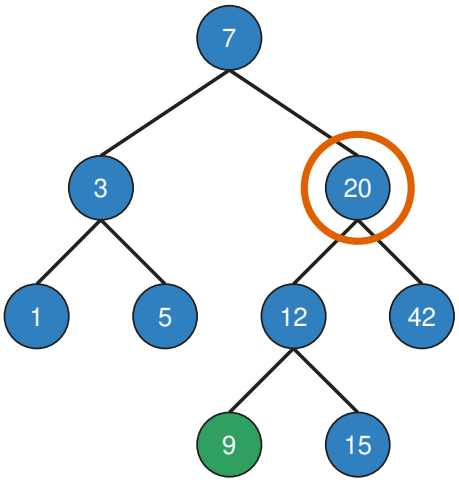
Search logic (looking for S)

- Looking for If *S* is equal to the current node, you found it
- If *S* is smaller than the current node, take the left branch
- If *S* is bigger than the current node, take the right branch
- If there are no branches, *S* was not found

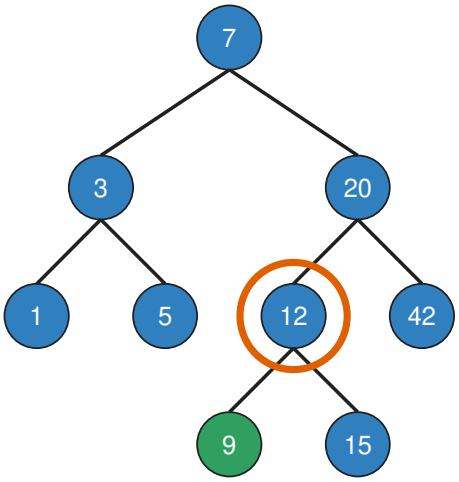
Searching for 9



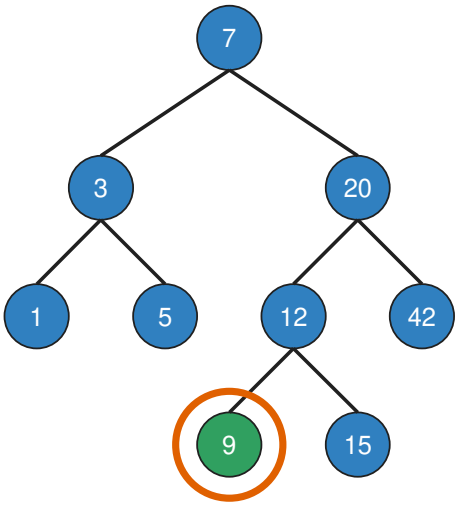
Searching for 9



Searching for 9



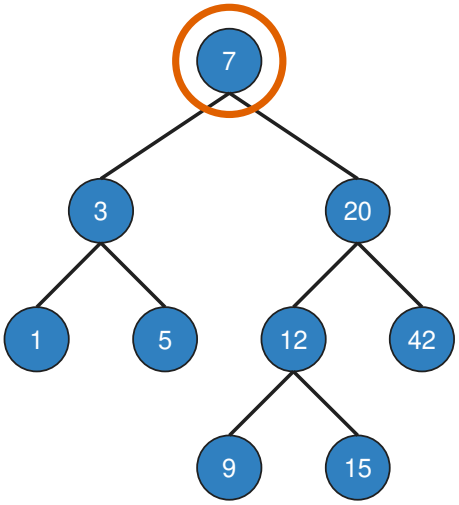
Searching for 9



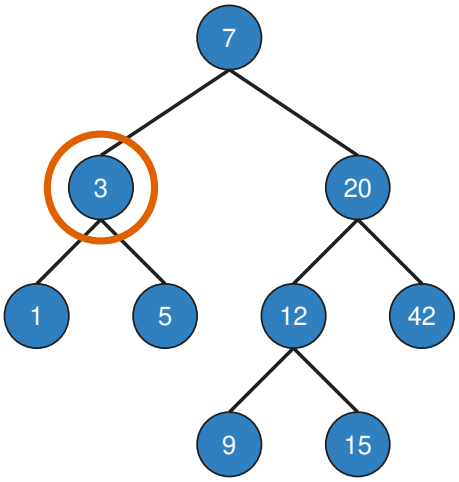
Inserting into the Tree

- Inserting is handled exactly like a search
- The only difference is that if the item is not found, the node is added
- If we reach a leaf…
  - we are already at the max-depth of the tree
  - we are at the node that needs to be updated
  - so, add a left or right node (based on value)
  - … wow, this is easy!

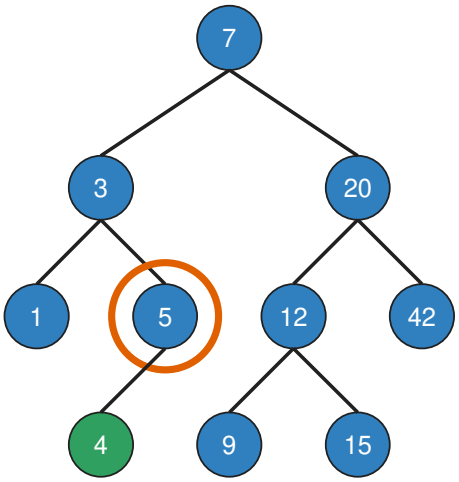
Inserting 4



Inserting 4



Inserting 4



Binary Search Trees vs. Arrays

- Insert into a Binary Search Tree
  - traverses down the tree
  - then add itself as a leaf
  - it requires only **O(log n)**
- Insert into an Array
  - must expanded when new elements are added
  - …and compacted when elements are removed
  - these requires **O(n)**

Deleting From a BST

Not as easy as Heaps

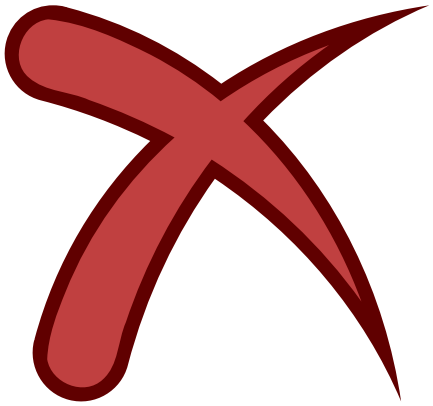
15

Deleting From a BST

- Deleting from a BST is a tad more tricky
- When the node is deleted, the tree needs to be re-linked to *still preserve the ordering*

16

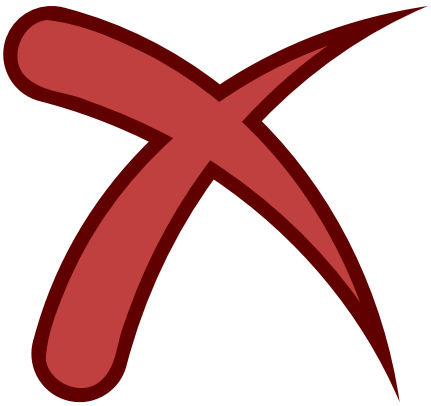
Deleting From a BST



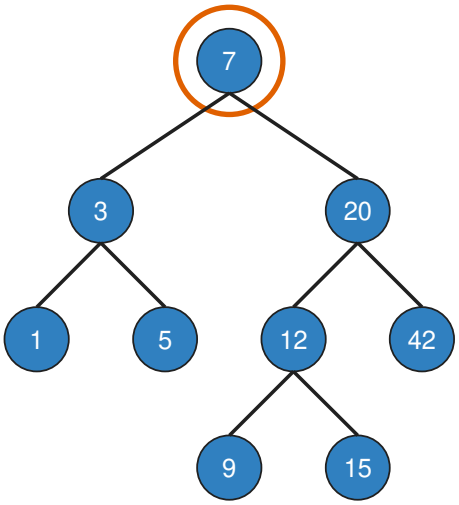
- Fortunately, while the logic might seem a tad hard, it is fairly straight forward
- There are 3 different scenarios that must be taken into account based on the node being deleted

Case 1: The Node is a Leaf

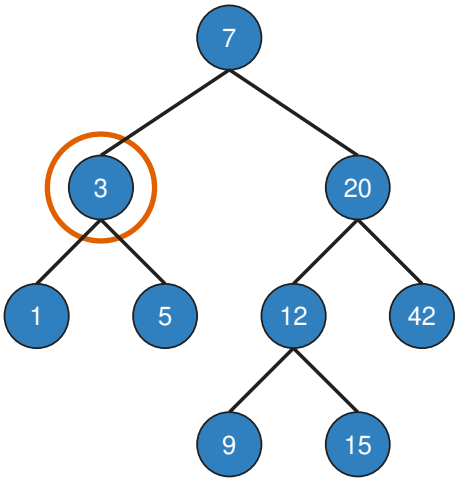
- If the node (to be deleted) is a leaf, it can simply be removed from the parent
- This is due to, the fact that, nothing is linked from the node



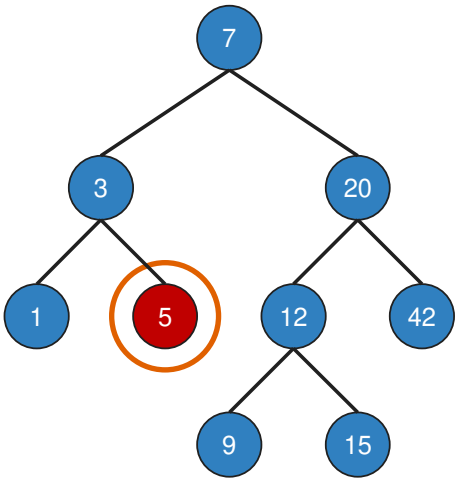
Case 1: Deleting 5



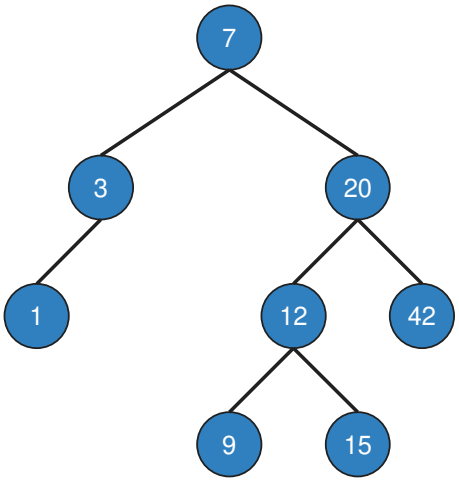
Case 1: Deleting 5



Case 1: Deleting 5

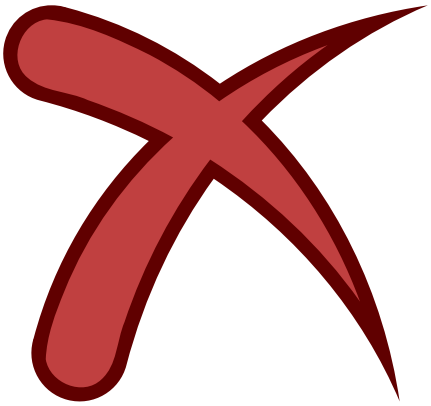


Case 1: Deleting 5

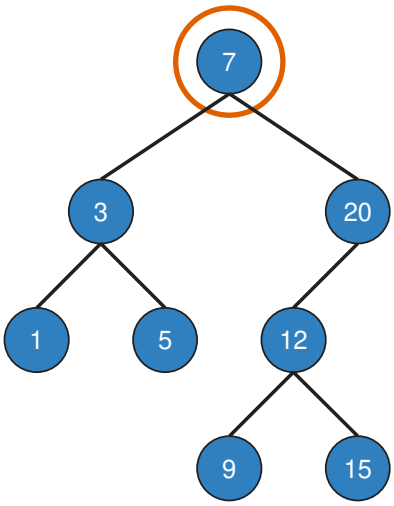


Case 2: One Child

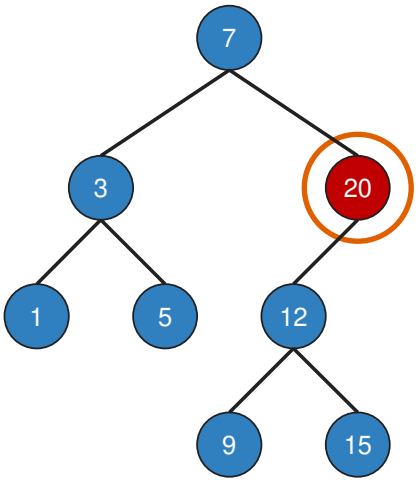
- If the deleted node has a single child, then that child can be *"promoted"*
- The deleted node's child is moved into its position
- … so, the subtree moves into the space created by the deleted node



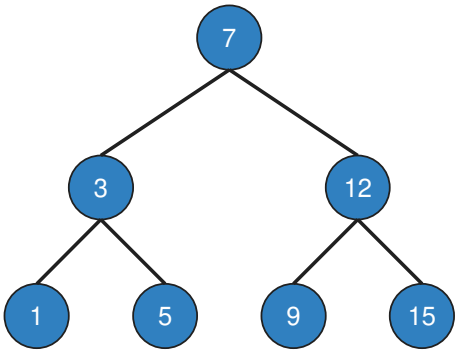
Case 2: Deleting 20



Case 2: Deleting 20

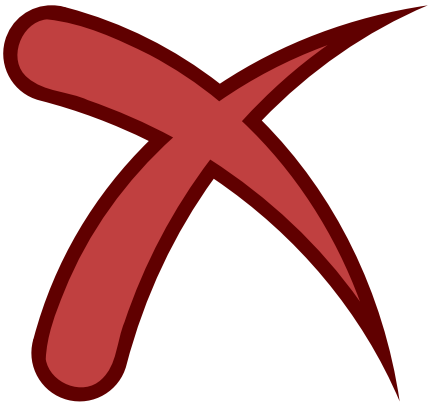


Case 2: Deleting 20



Case 3: Two Children

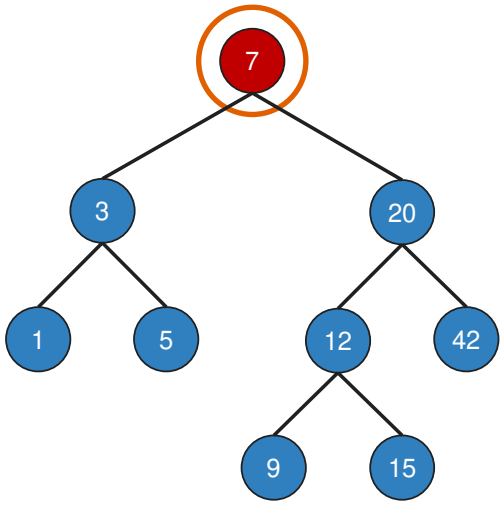
- However, if the deleted node has two children, then things get a tad more complex
- We need to find a node to *"promote"* to the deleted node's position



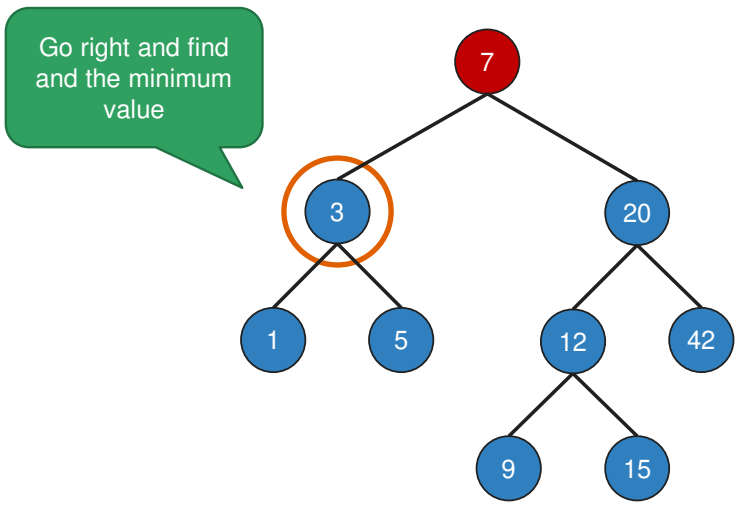
Case 3: Two Children

- We need to find a value that is mathematically next to the deleted value
- There are two equally valid options
- Choose one:
  - find the maximum node of the left (smaller) branch
  - … or find the minimum node of the right (larger) branch

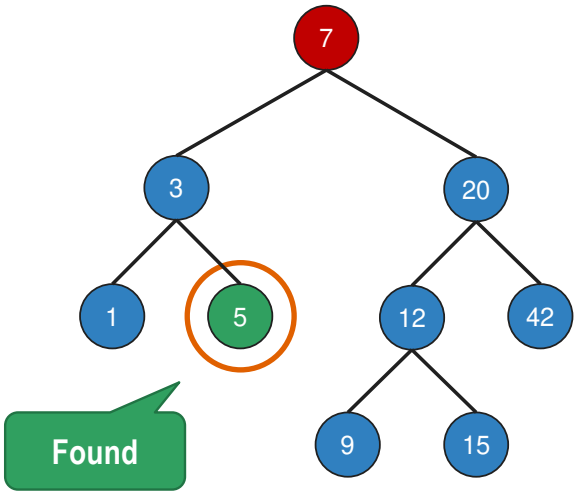
Case 3: Delete 7 – Max of Minimums



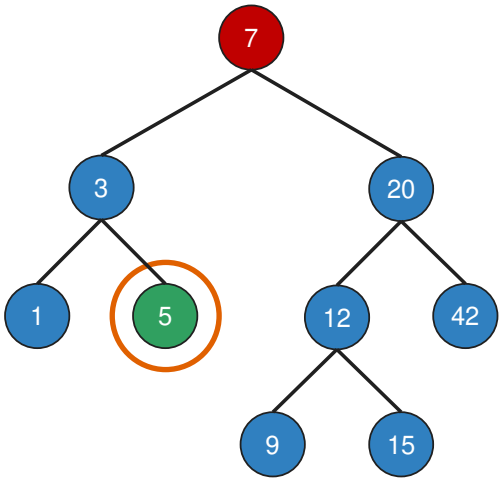
Case 3: Delete 7 (Going Right Once)



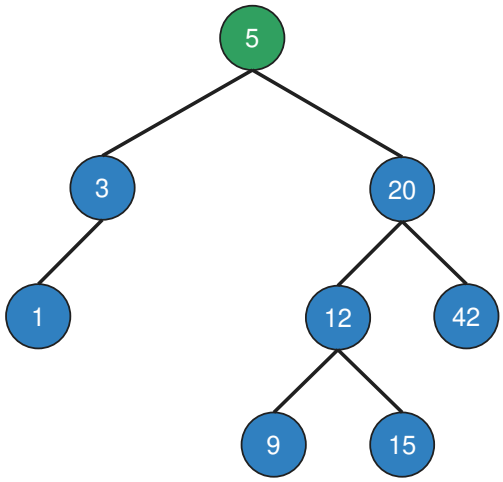
Case 3: Delete 7 (Keep Going Left)



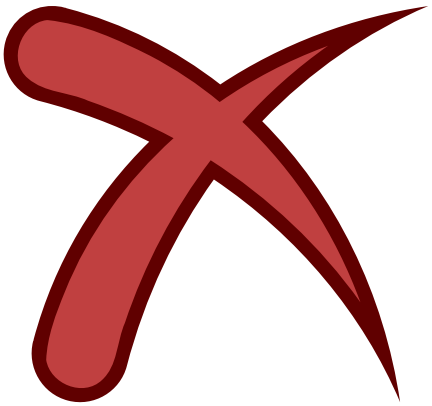
Case 3: Promote the Leaf



Case 3: Delete 7 Done

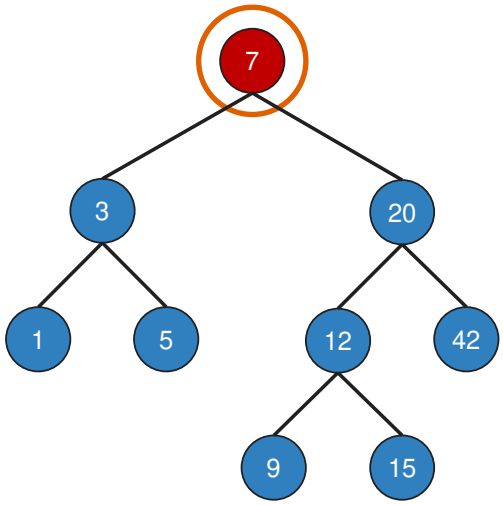


Case 3: Two Children

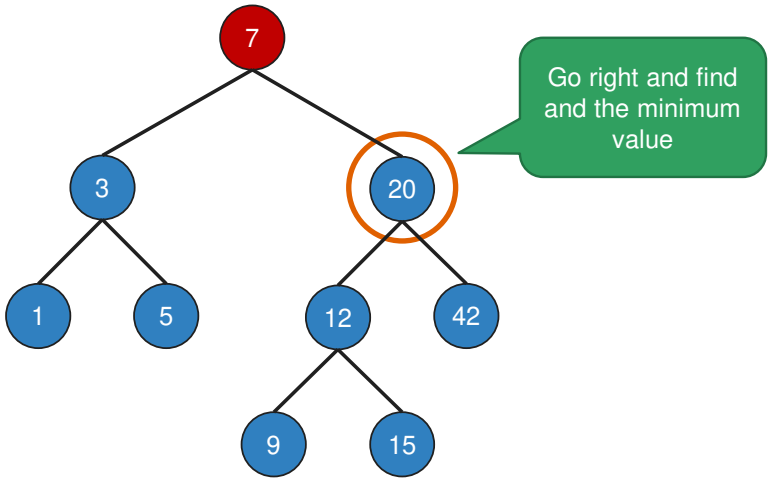


- It doesn't matter which approach you use
- As long as you find a value that fits mathematically into the old node's position it works.

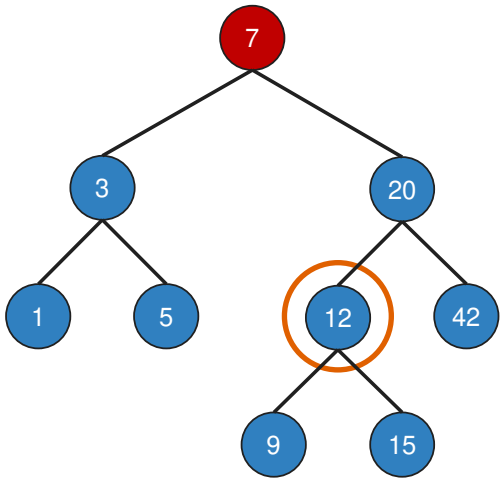
Case 3: Delete 7 – Min of Maximums



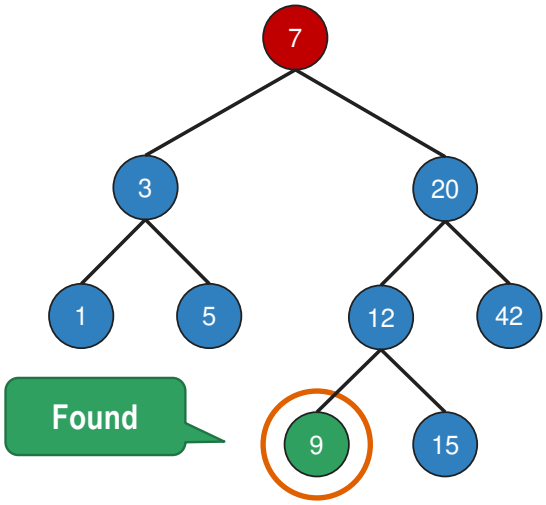
Case 3: Delete 7 (Going Right Once)



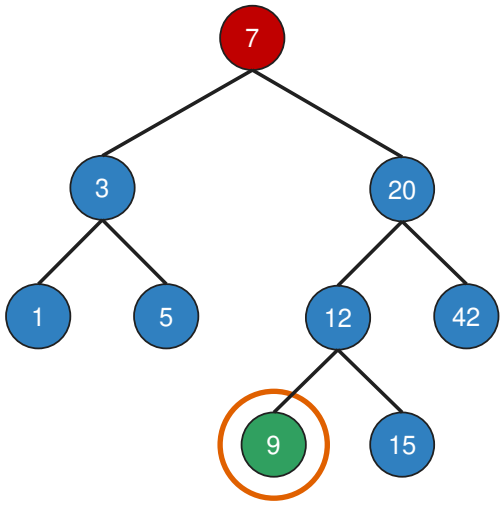
Case 3: Delete 7 (Keep Going Left)



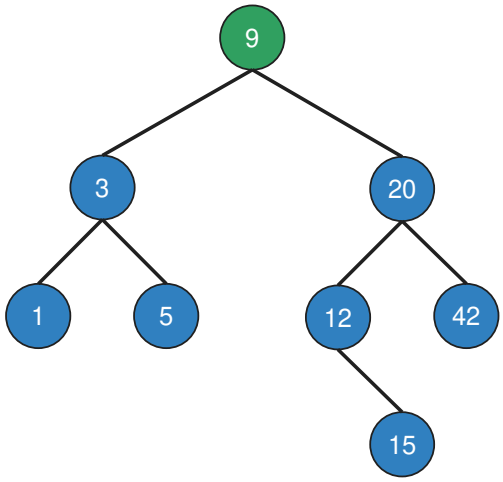
Case 3: Delete 7 (Keep Going Left)



Case 3: Promote the Leaf



Case 3: Deleting 7 - Done



The Problem with BSTs

Climbing a tree has never been so fun

41

The Problem with BSTs

- In a BST, internal nodes <u>never</u> change – and have a profound affect on the rest of the tree
- There are cases where the tree is unbalanced – one particular path contains most of the data

42

The Problem with BSTs

- This can easily occur of the data is not truly random (which is generally the case)
- When this happens, **the time complexity slowly deteriorates to O(n)**

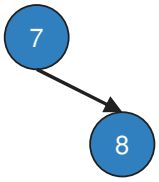
Let's Try Adding…

- Let's add the following number to a binary search tree
- How did this tree look?

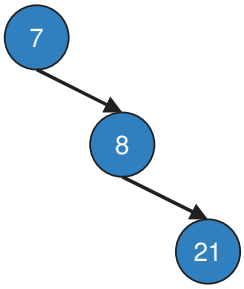
7, 8, 21, 43, 76

Let's add: 7, 8, 21, 43, 76

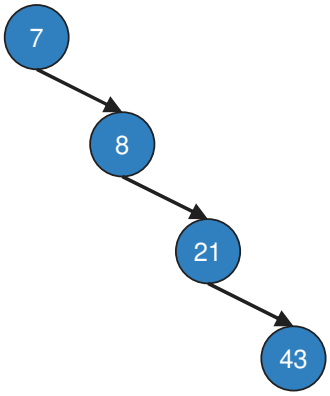
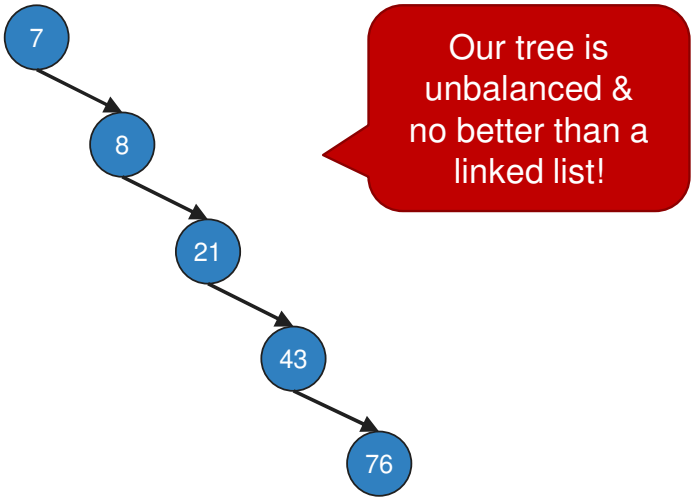
Let's add: 7, 8, 21, 43, 76



Let's add: 7, 8, 21, 43, 76



Let's add: 7, 8, 21, 43, 76



Let's add: 7, 8, 21, 43, 76